

ԿՈՒՐՍԱՅԻՆ ԱՇԽԱՏԱՆՔ

Առարկա՝	Օբյեկտ Կողմնորոշված Ծրագրավորում
Թեմա՝	Ծրագրավորման լեզվի կոմպիլյատորի և վիրտուալ մեքենայի մշակումը
Դասախոս՝	Խաչատրյան Տ.
Ուսանող՝	Արման Խանզադյան

1. Ներածություն
2. Լեզվի ճարտարապետություն
 - Lexer (լեքսիկական վերլուծիչ)
 - Parser (շարահյուսական վերլուծիչ)
 - AST (աբստրակտ շարահյուսական ծառ)
 - Symbol Table (սիմվոլների աղյուսակ / scope-եր)
3. Կոմպիլյատոր և միջանկյալ կոդ (IR / TAC)
4. Linker (կապակցիչ)
5. Վիրտուալ մեքենա (VM)
6. Debugger (վրիպագերծիչ)
7. Build համակարգ (Release և Debug)
8. Եզրակացություն
9. Հարցեր և պատասխաններ
10. Գրականություն

ՆԵՐԱԾՈՒԹՅՈՒՆ

Ծրագրավորման լեզուների մշակումը և կոմպիլյատորների կառուցումը հանդիսանում են ժամանակակից համակարգչային գիտությունների կարևորագույն ոլորտներից մեկը: Կոմպիլյատորը ծրագիր է, որը մի լեզվով գրված ելակոդը (source code) թարգմանում է ավելի ցածր մակարդակի հրամանների, որոնք կարող է կատարել մեքենան:

Սույն աշխատանքում C++17 լեզվով իրականացվել է փոքր C-նման լեզվի համար նախատեսված ամբողջական գործիքաշար՝

- **Compiler** (կոմպիլյատոր)
- **IR / TAC** (Three Address Code — եռհասցեանի միջանկյալ կոդ)
- **Linker** (կապակցիչ — սիմվոլների և relocation-ների լուծում)
- **Virtual Machine** (վիրտուալ մեքենա)
- **Debugger** (վրիպագերծիչ)

Մշակման հոսքագիծը (pipeline)՝

Source Code → Lexer → Parser → AST → IR (TAC) → LogicalCode → CodeGen → Optimizer → Linker → Assembler → ExecIO → Loader + VM → Արդյունք

Աշխատանքի նպատակն է ցույց տալ կոմպիլյատորի բոլոր դասական փուլերը՝ ելակոդից մինչև կատարումը սեփական վիրտուալ մեքենայի վրա:

1. ԼԵՉԿԻ ՃԱՐՏԱՐԱԴՊԵՏՈՒԹՅՈՒՆ

1.1 Lexer (լեքսիկական վերլուծիչ)

Lexer-ը կարդում է ելակոդը մեկ նիշ առ նիշ և բաժանում **token**-ների: Token-ը ամենափոքր իմաստ ունեցող միավորն է: Բացի այդ՝ բաց է թողնում բացատներն ու `//` մեկնաբանությունները և հաշվում տողերի համարները՝ սխալների հաղորդագրությունների համար: Ֆայլ՝ `include/Lexer.h`:

Token-ների տեսակները՝

- **keywords** → `int, float, void, if, else, while, do, for, return, break, continue, switch, case, default`
- **identifiers** (անուններ)
- **numbers** (թվեր)
- **operators** → `+ - * / %`, համեմատման՝ `== != < > <= >=`, տրամաբանական՝ `&& || !`
- **punctuation** → `( ) { } [ ] ; , .`

Օրինակ՝ `int a = 5;` դառնում է՝ `INT_KW, VARIABLE(a), ASSIGN, NUMBER(5), SEMICOLON`:

1.2 Parser (շարահյուսական վերլուծիչ)

Parser-ը token-ների հոսքից կառուցում է ծրագրի կառուցվածքը՝ ստուգելով քերականությունը: Կիրառվում է երկու մոտեցում՝

- **Recursive Descent** — ծրագրի կառուցվածքի համար (հայտարարություններ, օպերատորներ)
- **Expression parser** (precedence climbing) — օպերատորների առաջնահերթության (priority) համար, որպեսզի, օրինակ, `2 + 3 * 4` ճիշտ հաշվարկվի

Արդյունքը **AST**-ն է: Ֆայլ՝ `include/Parser.h`:

1.3 AST (Աբստրակտ Շարահյուսական Ծառ)

AST-ն ծառ է, որը ցույց է տալիս ծրագրի կառուցվածքը: Յուրաքանչյուր node ներկայացնում է լեզվական կառույց: Հիմնական node-երը (`NodeKind`)`

Node	Նշանակություն
PROGRAM	ամբողջ ծրագիրը
FUNC_DECL	ֆունկցիայի հայտարարություն
BLOCK	հրամանների բլոկ <code>{ ... }</code>
IF_STMT / WHILE_STMT / FOR_STMT	ճյուղավորում և ցիկլեր
VAR_DECL	փոփոխականի հայտարարություն
ASSIGN_EXPR	վերագրում <code>a = ...</code>
BINOP / UNOP	երկտեղ/միատեղ գործողություն
NUM_LIT	թվային հաստատուն
VAR_REF	փոփոխականի օգտագործում
CALL	ֆունկցիայի կանչ
RETURN_STMT	վերադարձ

Հիշողության ապահով կառավարման համար օգտագործվում է `std::unique_ptr` (smart pointer): Ֆայլ `include/Common.h` :

AST-ի օրինակ `a + b` -ի համար`

```
      BINOP "+"
      /    \
VAR_REF    VAR_REF
(a)        (b)
```

1.4 Symbol Table (սիմվոլների կառավարում / scope-եր)

Փոփոխականների և ֆունկցիաների անունները կառավարվում են scope-երով: IR գեներատորում օգտագործվում են ներդրված (nested) scope-եր`

`std::vector<std::map>` կառուցվածքով, ինչպես նաև գլոբալ փոփոխականների աղյուսակ: Հիմնական գործողությունները`

- փոփոխականի **հայտարարում** (declare) ընթացիկ scope-ում
- անվան **որոնում** (lookup) ներքին scope-ից դեպի դուրս
- ֆունկցիաների հասցեների պահպանում (symbol table գործարկվող ֆայլում)

Կողի գեներացիայի փուլում ֆունկցիաների անունները դառնում են **սիմվոլներ**, իսկ կանչերը` **relocation**-ներ, որոնք լրացվում են, երբ բոլոր հասցեները հայտնի են:

2. ԿՈՄՊԻԼՅԱՏՈՐ ԵՎ ՄԻՋԱՆԿՅԱԼ ԿՈԴ (IR / TAC)

2.1 IR գեներատոր (Three Address Code)

AST-ն վերածվում է **եռհասցեանի կոդի** (TAC) — պարզ հրամանների, որոնք ունեն ոչ ավելի, քան երեք օպերանդ: Հրամանի կառուցվածքը՝ `op, dst, src1, src2`:

Ֆայլեր՝ `src/IRGen.cpp`, `include/ir.h`:

Հիմնական օպ-երը՝

- թվաբանական՝ `ADD, SUB, MUL, DIV, MOD`
- համեմատում և ճյուղավորում՝ `CMP, BR_EQ, BR_NEQ, JMP`
- ֆունկցիաներ՝ `CALL, RET`, ստեղծ՝ `PUSH, POP`
- ֆունկցիայի frame՝ `PUSH_BP, SET_BP_SP, ALLOC_STACK, RESTORE_BP`

Օգտագործվում են՝

- **temp** ժամանակավոր փոփոխականներ → `t0, t1, t2 ...`
- **label**-ներ → `L0, L1 ...` control flow-ի համար

Օրինակ՝ `c = a + b` դառնում է՝

```
t0 = a + b
c  = t0
```

LogicalCode (տրամաբանական օպերատորների իշեցում)

Առանձին փուլ `&&`, `||`, `!` օպերատորները վերածվում է համեմատությունների և ճյուղավորումների, քանի որ դրանք ունեն կարճ-փակ (short-circuit) վարք: Ֆայլ՝

`src/LogicalCode.cpp`:

2.2 Optimizer (օպտիմիզատոր)

Կիրառվում է **peephole** օպտիմիզացիա — հրամանների փոքր պատուհանում ավելորդ կոդի հեռացում: Տվյալ դեպքում հեռացվում է «մեռած» գրառումը (dead store)՝ երբ երկու `MOV_CONST` իրար հետևից գրվում են նույն ռեգիստրը: Ֆայլ՝

`src/optimizer.cpp`:

Հնարավոր օպտիմիզացիաներ՝ Constant Folding, Copy Propagation, Dead Code Elimination: **Debug ռեժիմում** օպտիմիզացիան կարելի է անջատել, որպեսզի գեներացված կոդը մնա ընթերցելի և մոտ լինի սկզբնական IR-ին:

Assembler և ExecIO

Assembler-ը տպում է հրամանների ընթերցելի ցուցակը (disassembly): **ExecIO**-ն գրում է `.exec` գործարկվող ֆայլ՝ վերնագրով (signature `EXEC`) և բաժիններով՝ code, data, symbols, relocations, jump tables: Ֆայլեր՝ `src/assembler.cpp`,

`src/ExecIO.cpp`:

3. LINKER (ԿԱՊԱԿՑԻՉ)

Linker-ը (կապակցիչ) լուծում է սիմվոլային հղումները՝ վերածելով դրանք իրական հասցեների: Ֆայլեր՝ `include/linker.h`, `src/linker.cpp`:

3.1 Ինչու է անհրաժեշտ

Երբ կոդի գեներատորը արտադրում է ֆունկցիայի կանչ (`CALL`), կանչվող ֆունկցիայի հասցեն դեռ կարող է հայտնի չլինել (ֆունկցիան կարող է հայտարարված լինել ավելի ուշ): Այդ պատճառով գեներատորը թողնում է «լրացման տեղ» (placeholder) և գրանցում **relocation**՝

- **Symbol** — ֆունկցիայի անուն + հասցե (symbol table)
- **Relocation** — կանչի հրամանի համար + թիրախ ֆունկցիայի անուն

3.2 Ինչպես է աշխատում

Linker-ը անցնում է relocation-ների աղյուսակով, ամեն թիրախ որոնում է symbol table-ում ըստ անվան, և իրական հասցեն գրում (patch) կանչի հրամանի մեջ: Լուծումից հետո ծրագիրը այլևս չունի չլուծված հղումներ և պատրաստ է կատարման:

```
for (relocation r : relocs)
    addr = symbolTable[r.targetName]    // որոնում ըստ անվան
    code[r.instrIndex].rs1 = addr       // հասցեի լրացում
```

Եթե կանչվող ֆունկցիան գոյություն չունի, linker-ը այն հաշվում է որպես **unresolved reference** և հաղորդում սխալի մասին (`unresolvedCount()`), փոխանակ լուռ արտադրելու սխալ ծրագիր:

Կարևոր է, որ կապակցումը (linking) առանձին փուլ է կոդի գեներացիայից. դա համապատասխանում է իրական գործիքաչափերի կառուցվածքին, որտեղ կոմպիլյացիան և կապակցումը (compile և link) տարանջատված են:

4. ՎԻՐՏՈՒԱԼ ՄԵՔԵՆԱ (VM)

Վիրտուալ մեքենան ծրագիր է, որը նմանակում է պրոցեսորին. ունի ռեգիստրներ և հիշողություն և կատարում է սեփական հրամանների հավաքածուն ծրագրային ճանապարհով: Ֆայլեր՝ `include/vm.h`, `src/vm.cpp`:

4.1 Հիշողություն (Memory)

Մեկ 1 ՄԲ հասցեային տարածք՝ բաժանված հատվածների (segments)՝

- **Code Section** — հրամանները
- **Data Section** — գլոբալ հաստատուններ/տվյալներ
- **Heap Section** — դինամիկ հիշողություն
- **Stack Section** — ֆունկցիաների frame-ները (աճում է վերևից ներքև)

4.2 Պրոցեսոր (Processor)

Աշխատում է դասական ցիկլով՝ **Fetch → Decode → Execute**, մինչև `HALT` հրաման՝

1. **Fetch** — կարդալ հրամանը `pc` (ծրագրի հաշվիչ) հասցեից
2. **Decode** — որոշել օփ-ը և օպերանդները
3. **Execute** — կատարել, թարմացնել ռեգիստրները/հիշողությունը, առաջ տանել `pc`

VM-ն **ռեգիստրային** է և ունի՝

- 32 ռեգիստր (նաև 16-բիթանոց տեսք), word size՝ 16 / 32 / 64 բիթ
- Instruction Pointer / Program Counter (`pc`, `ip`)
- Stack Pointer (`sp`), Base Pointer (`bp`), Flags (`ff`)

4.3 Call Stack (կանչերի ստեկ)

Ֆունկցիայի կանչի ժամանակ պահպանվում է վերադարձի հասցեն և կանչողի base pointer-ը, ստեղծվում նոր frame, և կատարումը անցնում ֆունկցիային: `RET`-ի ժամանակ դրանք վերականգնվում են, արդյունքը դրվում է ռեգիստր 0-ում: Frame-ի կառուցվածքը պարունակում է՝

- վերադարձի հասցե (returnIP)
- կանչողի base pointer
- լոկալ փոփոխականներ (locals)

Օգտագործվող հրամաններ՝ `CALL`, `RET`, `PUSH_BP`, `SET_BP_SP`, `ALLOC_STACK`, `RESTORE_BP`, `LOAD_LOCAL`, `STORE_LOCAL`:

5. DEBUGGER (ՎՐԻՊԱՉԵՐԾԻՉ)

Debugger-ը թույլ է տալիս հետևել ծրագրի կատարմանը քայլ առ քայլ: Ֆայլեր՝ `include/debugger.h`, `src/debugger.cpp`:

5.1 Կառուցվածք

Կատարումը բաժանված է առանձին քայլերի՝ `runOne()` / `step()`, ինչը թույլ է տալիս կանգ առնել ցանկացած հրամանի վրա: Կա նաև **Debug build**, որը տպում է ամեն հրամանի **trace** (հետք)՝ ցույց տալով `pc`-ն, `op`-ը և օպերանդները, որպեսզի երևա `fetch-decode-execute` ցիկլը՝

```
[trace] pc=0    PUSH_BP    rd=0  rs1=0  rs2=0
[trace] pc=2    MOV_CONST  rd=1  rs1=0  rs2=0
[trace] pc=3    ADD        rd=2  rs1=1  rs2=0
...
Result R0 = 15 (expected 15)
```

5.2 Breakpoints (կանգառի կետեր)

Կանգառ կարելի է դնել ըստ հրամանի համարի (PC): Կողմ ունի նաև ֆունկցիաների `symbol table`, ինչը հնարավորություն է տալիս կապել կանգառները ֆունկցիայի անվան հետ:

5.3 Հրամաններ (REPL)

Ինտերակտիվ ռեժիմում (`--debug`) հասանելի են հրամանները՝

Հրաման	Գործողություն
<code>s</code> (step)	կատարել մեկ հրաման
<code>c</code> (continue)	շարունակել մինչև breakpoint
<code>r</code> (registers)	ցույց տալ ռեգիստրները
<code>m</code> (memory)	ցույց տալ հատվածները (segments)
<code>b N</code> (break)	դնել breakpoint PC=N-ում
<code>q</code> (quit)	դուրս գալ

6. BUILD ՀԱՄԱԿԱՐԳ (Release և Debug)

Նախագիծը հավաքվում է երկու ստանդարտ կոնֆիգուրացիայով՝ և՛ `Makefile`-ով, և՛ `CMake`-ով:

Կոնֆիգուրացիա	Դրոշներ	Արդյունք
Release	<code>-O2 -DNDEBUG</code> (օպտիմիզացված)	<code>build/release/compiler</code>
Debug	<code>-O0 -g3 -DDEBUG_BUILD</code> + sanitizer-ներ + gdb սիմվոլներ	<code>build/debug/compiler</code>

Հրամանները՝

```
make           # Release
make debug     # Debug (trace + sanitizers)
make run       # հավաքել և գործարկել Release
make run-debug # հավաքել և գործարկել Debug
```

Debug build-ը սահմանում է `DEBUG_BUILD` մակրոն, որն ակտիվացնում է վիրտուալ մեքենայի հրահանգ առ հրահանգ trace-ը:

Ցուցադրական ծրագիր

```
int main() {
    int a = 5;
    int b = 10;
    int c = 0;
    if (a < b) {
        c = a + b;
    }
    return c;
}
```

Քանի որ `a < b` ճշմարիտ է, `c = 15`, և մեքենան վերադարձնում է **R0 = 15**:

ԵԶՐԱԿԱՑՈՒԹՅՈՒՆ

Աշխատանքի ընթացքում C++17 լեզվով իրականացվել է ամբողջական համակարգ, որը ներառում է կոմպիլյատորի բոլոր դասական փուլերը՝

- Lexer (լեքսիկական վերլուծիչ)
- Parser (շարահյուսական վերլուծիչ)
- AST (աբստրակտ շարահյուսական ծառ)
- IR / TAC (եռհասցեանի միջանկյալ կոդ)
- Optimizer (օպտիմիզատոր)
- Linker (կապակցիչ)
- Virtual Machine (վիրտուալ մեքենա)
- Debugger (վրիպագերծիչ)

Բոլոր մոդուլներն աշխատում են կայուն. ցուցադրական ծրագիրը հաջողությամբ կոմպիլյացվում և կատարվում է՝ վերադարձնելով ճիշտ արդյունքը (15)՝ բոլոր ռեժիմներում (սովորական, `--exec`, `--debug`): Ավելացվել է նաև ստանդարտ Debug build՝ հրահանգների trace-ով և sanitizer-ներով, ինչը հեշտացնում է սխալների հայտնաբերումը:

ՀԱՐՑԵՐ ԵՎ ՊԱՏՎԱՍԽԱՆՆԵՐ

Հնարավոր հարցեր, որոնք կարող է տալ դասախոսը, և դրանց պատասխանները:

Հ1. Ի՞նչ է կոմպիլյատորը:

Ծրագիր է, որը մի լեզվով գրված ելակոդը թարգմանում է ավելի ցածր մակարդակի հրամանների, որոնք կարող է կատարել մեքենան: Մեր դեպքում՝ փոքր C-նման լեզուն վերածվում է վիրտուալ մեքենայի հրամանների:

Հ2. Ո՞րն է կոմպիլյատորի և ինտերպրետատորի տարբերությունը:

Կոմպիլյատորը նախ թարգմանում է ամբողջ ծրագիրը և արտադրում արդյունք (հրամաններ/ֆայլ), որը գործարկվում է հետո: Ինտերպրետատորը ուղղակիորեն կատարում է ելակոդը՝ տող առ տող:

Հ3. Որո՞նք են կոմպիլյատորի փուլերը:

Լեքսիկական վերլուծություն (Lexer), շարահյուսական վերլուծություն (Parser), միջանկյալ կոդի գեներացիա (IR/TAC), օպտիմիզացիա, կոդի գեներացիա և կապակցում (Linker):

Հ4. Ի՞նչ է token-ը:

Ելակոդի ամենափոքր իմաստ ունեցող միավորը՝ բանալի բառ, անուն, թիվ կամ օպերատոր:

Հ5. Ի՞նչ է AST-ն:

Աբստրակտ շարահյուսական ծառ՝ ծրագրի կառուցվածքի ծառային ներկայացում, որտեղ ամեն node լեզվական կառույց է:

Հ6. Ի՞նչ է Three Address Code-ը (TAC):

Միջանկյալ կոդ, որի յուրաքանչյուր հրաման ունի ոչ ավելի, քան երեք օպերանդ, օրինակ՝ `t0 = a + b`: Հեշտ է օպտիմիզացնել և թարգմանել:

Հ7. Ինչու՞ են `&&`, `||`, `!` առանձին իջեցվում:

Որովհետև ունեն կարճ-փակ (short-circuit) վարք, ուստի վերածվում են համեմատությունների և ճյուղավորումների, ոչ թե մեկ թվաբանական հրամանի:

Հ8. Ի՞նչ է linker-ը և ունի՞ր այս նախագիծը:

Այո: Linker-ը լուծում է սիմվոլային հղումները՝ relocation-ների և symbol table-ի միջոցով գտնում է կանչվող ֆունկցիայի հասցեն և գրում `CALL` հրամանի մեջ: Ֆայլ՝ `src/linker.cpp`:

Հ9. Ինչու՞ է կապակցումը (linking) առանձին քայլ:

Քանի որ `CALL` գեներացնելիս ֆունկցիայի հասցեն դեռ կարող է հայտնի չլինել, գեներատորը թողնում է placeholder և գրանցում relocation: Երբ բոլոր հասցեները հայտնի են, linker-ը լրացնում է դրանք:

Հ10. Ի՞նչ է վիրտուալ մեքենան:

Ծրագիր, որը նմանակում է պրոցեսորին. ունի ռեգիստրներ ու հիշողություն և կատարում է սեփական հրամանները ծրագրային ճանապարհով:

Հ11. Ի՞նչ է **fetch-decode-execute** ցիկլը:

Պրոցեսորի հիմնական ցիկլը՝ կարդալ հրամանը (fetch), վերծանել (decode), կատարել (execute), անցնել հաջորդին: Կրկնվում է մինչև `HALT`:

Հ12. Ձեր VM-ն ռեգիստրայի՞ն է, թե՞ ստեկային:

Ռեգիստրային է. գործողությունները կատարվում են համարակալված ռեգիստրների վրա (32 ռեգիստր): Ստեկային մեքենան (օրինակ՝ JVM) օպերանդները պահում է ստեկում:

Հ13. Ո՞րն է **Release** և **Debug build**-երի տարբերությունը:

Release-ը օպտիմիզացված է (`-O2`) սովորական օգտագործման համար: Debug-ը (`-O0 -g3 -DDEBUG_BUILD`) ունի սիմվոլներ, sanitizer-ներ և հրահանգների trace՝ սխալների հայտնաբերման համար:

Հ14. Ի՞նչ է վերադարձնում **ցուցադրական ծրագիրը** և ինչու:

Վերադարձնում է 15. `a=5`, `b=10`, և քանի որ `a<b` ճշմարիտ է՝ `c=a+b=15`:

ԳՐԱԿԱՆՈՒԹՅՈՒՆ

1. Aho, Lam, Sethi, Ullman — “Compilers: Principles, Techniques, and Tools”
2. Andrew W. Appel — “Modern Compiler Implementation”
3. Bjarne Stroustrup — “The C++ Programming Language”
4. cppreference.com — C++ ստանդարտ գրադարանի փաստաթղթեր